

Docker, CI/CD & AWS Deployment — Revision Guide

Inventory Management Project

Hands-on reference · Phases 1–5 (through automated EC2 deployment)

- [1. The Big Picture](#)
- [2. Docker Fundamentals](#)
 - [Images vs Containers](#)
 - [Essential commands](#)
 - [The Dockerfile](#)
 - [Multi-stage builds \(the real Dockerfile\)](#)
 - [ENTRYPOINT vs CMD \(an important gotcha\)](#)
 - [Layers & the build cache](#)
- [3. Volumes — persisting data](#)
- [4. Networks — how containers find each other](#)
- [5. Docker Compose](#)
- [6. Container Registries \(Docker Hub vs ECR\)](#)
- [7. The Spring Boot App & Profiles](#)
- [8. CI/CD Concepts](#)
- [9. GitHub Actions — workflow anatomy](#)
- [10. AWS — ECR & IAM](#)
- [11. AWS — EC2 provisioning](#)
- [12. Deploying the stack on EC2](#)
- [13. Automated deployment \(SSH deploy job\)](#)
- [14. Quick command reference](#)
- [15. What's next — Kubernetes \(Phase 6\)](#)

1. The Big Picture

This guide recaps everything built for the **inventory-management** Spring Boot app: from Docker fundamentals, through a GitHub Actions CI/CD pipeline, to a self-managed deployment on an EC2 instance running nginx + the Java app + Redis + Postgres.

The full flow you built:

Your Mac (git push)



GitHub Actions — test → build image → push to Amazon ECR



SSH into EC2 → docker compose pull + up -d



Internet :80 → nginx → inventory-app :8080 →

├	Postgres :5432	(data)
└	Redis :6379	(cache)

Two mental models to keep:

- **Dockerfile = recipe to build ONE image.** Used in every environment.
 - **docker-compose / Kubernetes = orchestration** — what containers run and how they wire together. They *use* the image; they don't replace the Dockerfile.
 - Environments differ by **configuration (Spring profiles + env vars)**, not by different images. One image runs everywhere.
-

2. Docker Fundamentals

Images vs Containers

- **Image** = an immutable, packaged snapshot of your app + its runtime (a *class*).
- **Container** = a running instance of an image (an *object*).
- A container's filesystem is **ephemeral** — remove the container and anything written inside it is gone (unless stored in a volume).

Essential commands

Command	What it does
<code>docker build -t name:tag .</code>	Build an image from the Dockerfile in <code>.</code>
<code>docker images</code>	List local images
<code>docker run -d -p 80:8080 name</code>	Create + start a container (detached, port-mapped)
<code>docker ps</code> / <code>docker ps -a</code>	List running / all containers
<code>docker logs <name></code>	View a container's stdout/stderr
<code>docker exec -it <name> sh</code>	Open a shell inside a running container
<code>docker stop <name></code> / <code>docker rm <name></code>	Stop / remove a container
<code>docker pull <image></code>	Download an image from a registry
<code>docker push <image></code>	Upload an image to a registry
<code>docker image prune -f</code>	Delete dangling image layers (cleanup)

Common `docker run` flags:

```
docker run -d \           # -d = detached (background)
  --name inventory-app \   # a name (also its hostname on a network)
  --network inventory-net \# attach to a user-defined network
  -p 8080:8080 \          # publish hostPort:containerPort
  -e KEY=value \          # set an environment variable
  -v mydata:/path \       # mount a volume
  myimage:latest
```

The Dockerfile

A Dockerfile is the **build recipe**. Each instruction creates a layer.

```
FROM eclipse-temurin:17-jre # base image
WORKDIR /app                # working directory inside the image
COPY app.jar app.jar        # copy files in
EXPOSE 8080                  # documents the port (does not publish it)
ENTRYPOINT ["java", "-jar", "app.jar"] # what runs when the container starts
```

Multi-stage builds (the real Dockerfile)

Your production Dockerfile has **two stages**. This is what keeps the final image small and secure.

```
# ----- STAGE 1: build (has Maven + JDK + source) -----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B      # cache dependencies as a layer
COPY src ./src
RUN mvn package -B -DskipTests      # produce the jar

# ----- STAGE 2: runtime (fresh, JRE only) -----
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar  # copy ONLY the jar across
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why it matters:

- **AS build** **names** the stage so the second stage can reference it (**COPY --from=build**) and you can target it (**docker build --target build**).
- The second **FROM** starts a **brand-new image** (JRE only — no Maven, no compiler, no source).
- Only the jar crosses over. Everything else is thrown away → final image ~250 MB instead of ~800 MB, and no build tools or source shipped to production.
- **-DskipTests** skips tests *during the image build* (they already ran in CI). It does **not** strip the image — the multi-stage design does.

ENTRYPOINT vs CMD (an important gotcha)

- **CMD** — default command; **replaced** by anything you put after the image name in **docker run** .
- **ENTRYPOINT** — fixed command; arguments after the image name are **appended** to it, not replaced.

Gotcha we hit: `docker run cicc:latest mvn test` did **not** run tests — it ran `java -jar app.jar mvn test` (the `mvn test` was appended to the ENTRYPOINT and ignored). To run tests in a container you use the *build stage* or a Maven image, not the production image (which has no Maven anyway).

Layers & the build cache

- Each Dockerfile instruction is a cached layer. If nothing above a layer changed, Docker reuses it.
- That's why `COPY pom.xml` + `RUN mvn dependency:go-offline` comes **before** `COPY src` — dependencies only re-download when `pom.xml` changes, not on every code edit.

3. Volumes — persisting data

A **volume** is storage that lives *outside* the container, so data survives the container being stopped, removed, or rebuilt.

```
# named volume — Docker manages the storage (used for the Postgres DB)
docker run -d --name postgres -v pgdata:/var/lib/postgresql/data postgres:16-alpine

# bind mount — map a host path into the container (used for the nginx config)
docker run -d --name nginx -v ~/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
nginx:alpine
```

	Without a volume	With a volume
Data location	Inside the container	Outside / managed by Docker
Survives <code>rm</code> / rebuild?	❌ Lost	✅ Persists
Typical use	Stateless app	Database, uploads, config, logs

- Format is always `host-or-volume : container-path`. Append `:ro` for read-only.
 - The Java app needs **no** volume (stateless — state lives in Postgres). Postgres needs one so data isn't wiped on restart.
-

4. Networks — how containers find each other

A **user-defined network** gives containers DNS: they reach each other **by container name**.

```
docker network create inventory-net

docker run -d --name postgres --network inventory-net postgres:16-alpine
docker run -d --name inventory-app --network inventory-net \
  -e DB_URL=jdbc:postgresql://postgres:5432/inventorydb ...
```

- `postgres` in the JDBC URL is the **container name**, resolved by the network's built-in DNS. No IP addresses hardcoded.
 - This is why nginx can `proxy_pass http://inventory-app:8080` and the app can reach `redis:6379`.
 - Note: from *inside* a container, `localhost` means the container itself — not the host and not other containers. That's why we use service names, not `localhost`.
-

5. Docker Compose

Compose describes a **multi-container stack** in one declarative file, so you start everything with one command.

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: inventorydb
      POSTGRES_USER: inventory
      POSTGRES_PASSWORD: ${DB_PASSWORD}  # from the .env file
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

  inventory-app:
    image: ${ECR_IMAGE}
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_URL: jdbc:postgresql://postgres:5432/inventorydb
      REDIS_HOST: redis
    depends_on: [postgres, redis]

  nginx:
    image: nginx:alpine
    ports: ["80:80"]
    volumes:
      - /home/ec2-user/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [inventory-app]

volumes:
  pgdata:
```

Key commands:

```
docker compose up -d      # build/pull + start the whole stack (reads .env
                        automatically)
docker compose ps         # status of all services
docker compose pull       # fetch newer images
docker compose up -d      # recreates only the containers whose image changed
docker compose down       # stop + remove containers (keeps named volumes)
docker compose down -v    # also delete volumes (wipes data)
```

- Compose **auto-reads** a `.env` file in the current directory — no `--env-file` needed when it's named `.env`.
- `depends_on` sets start order (not readiness — for that you add a healthcheck).
- Compose **uses your Dockerfile** (`build: .`) or a prebuilt `image:` — it does not replace the Dockerfile.

Install on Amazon Linux 2023 (the plugin isn't bundled):

```
mkdir -p ~/.docker/cli-plugins
curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-
linux-x86_64 \
-o ~/.docker/cli-plugins/docker-compose
chmod +x ~/.docker/cli-plugins/docker-compose
```

6. Container Registries (Docker Hub vs ECR)

A **registry** is where images are stored so different machines can share them. The GitHub runner *builds* an image, but it's ephemeral — so it **pushes** to a registry, and the deploy target **pulls** from it.

```
GitHub runner: build → PUSH → Registry ← PULL ← EC2
                (temporary)      (persistent)      (deploy target)
```

- **Docker Hub** — the default public registry. Fine, but has pull rate limits and lives outside AWS.
- **Amazon ECR** — AWS's private registry. Best when deploying to AWS: same cloud, IAM auth, no rate limits. **You chose ECR.**

Private registries require authentication before any pull/push:

```
aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin \
281535664028.dkr.ecr.ap-south-1.amazonaws.com
```

- `get-login-password` returns a temporary token; it's piped into `docker login`.
- After login, `docker pull` / `docker push` work. `docker run` will **auto-pull** a missing image — but only *after* you're authenticated.

Gotcha: when a tag like `:latest` is reused, `docker run` uses the **local** copy if present — it won't fetch the new one. You must `docker pull` explicitly to update. (This is why the automated deploy runs `docker compose pull` before `up -d`.)

7. The Spring Boot App & Profiles

The single image behaves differently per environment via **Spring profiles + environment variables** — no code or image changes.

Profile	Database	Cache	When
default	H2 (in-memory)	in-memory	local dev, tests
prod	PostgreSQL	Redis	production / EC2

`application-prod.properties` reads everything from env vars with fallbacks:

```
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/inventorydb}
spring.datasource.username=${DB_USERNAME:inventory}
spring.datasource.password=${DB_PASSWORD:changeme}
spring.jpa.hibernate.ddl-auto=${DDL_AUTO:validate}
spring.cache.type=redis
spring.data.redis.host=${REDIS_HOST:localhost}
```

- `${DB_URL:default}` = “use env var `DB_URL`, else this default.”
- Activated with `-e SPRING_PROFILES_ACTIVE=prod`.
- `ddl-auto`: `validate` in real prod (never mutate schema); we override `DDL_AUTO=update` for a fresh empty DB so Hibernate creates the tables once.
- **Redis caching**: `@EnableCaching` + `@Cacheable(value="items", key="#id")` on reads, `@CacheEvict` on writes. Cache key in Redis looks like `items:1`. Cached entities must be `Serializable`.

8. CI/CD Concepts

- **CI (Continuous Integration)**: every push automatically builds + tests on a clean machine, so problems surface immediately.
- **CD (Continuous Deployment)**: after tests pass, automatically build + ship the artifact.

The gate principle: tests run *first*; the image is built/deployed *only if tests pass*. You don't build then test — you prove the code good, then package it.

Why we left Travis CI: it no longer offers a free tier (paid plans only), so we moved to **GitHub Actions** — free for public repos, built into GitHub, no external activation.

9. GitHub Actions — workflow anatomy

Vocabulary:

```
Workflow (the .yaml file)
└ triggered by an Event (on:)
  └ runs Jobs (each on its own fresh VM)
    └ each Job runs Steps in order
      └ a Step is a prebuilt Action (uses:) OR a shell command (run:)
```

Your pipeline has three jobs, chained with `needs: :`

```

on:
  push:
    branches: [main, 'feature/**']

jobs:
  test:                                     # JOB 1 – the gate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4           # clone the repo
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17', cache: maven }
      - run: mvn -B clean verify           # run the 34 tests

  build-and-push:                           # JOB 2 – only if test passed
    needs: test
    runs-on: ubuntu-latest
    env: { AWS_REGION: ap-south-1, ECR_REPOSITORY: inventory-management }
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }
      - id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }:${
            github.sha }
            ${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }:latest
          cache-from: type=gha               # restore cached Docker layers
          cache-to: type=gha,mode=max       # save layers back (mode=max = multi-stage too)

  deploy:                                   # JOB 3 – only if image was pushed
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.EC2_HOST }
          username: ec2-user
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd ~/app
            aws ecr get-login-password --region ap-south-1 | docker login --username AWS --
            password-stdin 281535664028.dkr.ecr.ap-south-1.amazonaws.com
            docker compose pull
            docker compose up -d
            docker image prune -f

```

The pieces explained:

- **on:** — what triggers the run (push to `main` or any `feature/**`).

- `runs-on: ubuntu-latest` — each job gets its **own fresh VM**; jobs do **not** share a filesystem.
- `needs:` — the gate. `test → build-and-push → deploy`, each waits for the previous to succeed.
- `uses:` — a prebuilt marketplace Action; `with:` = its inputs.
- `run:` — a shell command.
- `${{ secrets.X }}` — injects an encrypted secret (set under repo Settings → Secrets and variables → Actions).
- `${{ steps.login-ecr.outputs.registry }}` — reads an output from a previous step (needs that step to have an `id`).
- `${{ github.sha }}` — the commit SHA, used as an image tag for traceability.

Docker layer caching (why builds got 3x faster): `cache: maven` only speeds the *test* job. The *build* job runs Maven **inside** the Docker build, and GitHub runners have no Docker cache by default — so it re-downloaded dependencies every run. Adding `docker/build-push-action` with `type=gha` cache reuses the dependency layer across runs (as long as `pom.xml` is unchanged), dropping the build from ~2m to ~47s.

10. AWS — ECR & IAM

Create the registry: ECR → Create repository → `inventory-management` (private), region `ap-south-1`.

IAM for the CI user (push): the pipeline needs to *push*. The minimal policy must include the push actions (a read-only policy fails mid-push):

```
{
  "Effect": "Allow",
  "Action": [
    "ecr:GetAuthorizationToken",
    "ecr:BatchCheckLayerAvailability",
    "ecr:GetDownloadUrlForLayer",
    "ecr:BatchGetImage",
    "ecr:InitiateLayerUpload",
    "ecr:UploadLayerPart",
    "ecr:CompleteLayerUpload",
    "ecr:PutImage"
  ],
  "Resource": "*"
}
```

(The last four are the push actions. Or attach the managed policy

`AmazonEC2ContainerRegistryPowerUser`.)

- CI authenticates with an **access key + secret**, stored as GitHub secrets `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`.
 - Region and repo name are **not secret** — they live in the workflow as plain text.
-

11. AWS — EC2 provisioning

Launch: EC2 → Amazon Linux 2023, instance type (t2.micro free-tier for Docker; t3.small for Kubernetes), download the `.pem` key pair.

Security group (inbound rules):

Type	Port	Source	Why
SSH	22	My IP (or 0.0.0.0/0 for CI SSH)	Shell access
HTTP	80	Anywhere	nginx
Custom TCP	8080	My IP	Test the app directly

IAM instance role (pull): attach a role with `AmazonEC2ContainerRegistryReadOnly` to the instance (Actions → Security → Modify IAM role). The box then pulls from ECR with **temporary, auto-rotating credentials** — **no access keys stored on it**. (CI *pushes* with keys; the instance only *pulls* with a role.)

Connect:

```
chmod 400 ~/Downloads/your-key.pem          # SSH refuses world-readable keys
ssh -i ~/Downloads/your-key.pem ec2-user@<PUBLIC_IP>  # ec2-user = AL2023 default
```


12. Deploying the stack on EC2

Install Docker (Amazon Linux 2023):

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl enable --now docker      # start now + on every boot
sudo usermod -aG docker ec2-user        # run docker without sudo (re-login after)
```

Install the AWS CLI (if `aws --version` fails):

```
sudo dnf install -y unzip
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install
```

Run the stack (the manual version, before Compose):

```
docker network create inventory-net

# Postgres (with a volume for persistence)
docker run -d --name postgres --network inventory-net \
  -e POSTGRES_DB=inventorydb -e POSTGRES_USER=inventory -e POSTGRES_PASSWORD=inventory \
  -v pgdata:/var/lib/postgresql/data postgres:16-alpine

# Redis (cache - no volume needed)
docker run -d --name redis --network inventory-net redis:7-alpine

# Authenticate + run the app from ECR
aws ecr get-login-password --region ap-south-1 \
  | docker login --username AWS --password-stdin 281535664028.dkr.ecr.ap-south-1.amazonaws.com

docker run -d --name inventory-app --network inventory-net -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e DB_URL=jdbc:postgresql://postgres:5432/inventorydb \
  -e DB_USERNAME=inventory -e DB_PASSWORD=inventory -e DDL_AUTO=update \
  -e REDIS_HOST=redis -e REDIS_PORT=6379 \
  281535664028.dkr.ecr.ap-south-1.amazonaws.com/inventory-management:latest
```

nginx reverse proxy — config (`~/nginx/default.conf`):

```

server {
    listen 80;
    server_name _;
    location / {
        proxy_pass http://inventory-app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

```

docker run -d --name nginx --network inventory-net -p 80:80 \
-v ~/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine

```

- `proxy_pass http://inventory-app:8080` — forwards to the app by container name.
- nginx owning port 80 is the foundation for adding HTTPS later.

Verify caching works:

```

curl http://localhost/api/v1/inventory/1      # first call = cache miss (loads from DB)
curl http://localhost/api/v1/inventory/1      # second call = served from Redis
docker exec -it redis redis-cli KEYS '*'      # -> items::1

```

13. Automated deployment (SSH deploy job)

The `deploy` job (Section 9) SSHes into EC2 and redeploys. Setup:

1. **Security group:** allow SSH (22) from `0.0.0.0/0` (GitHub runners have dynamic IPs). Safe-ish because EC2 uses **key-only** auth. Production alternatives: AWS SSM or a self-hosted runner.
2. **GitHub secrets:** `EC2_HOST` (public IP) and `EC2_SSH_KEY` (full `.pem` contents).
3. The job runs on the box: `docker compose pull` → `docker compose up -d`.

Proven end-to-end: editing one file (adding `/api/v1/ping`) → push → tests → image built + pushed → runner SSHes in → app recreated with the new image. `curl http://<EC2>/api/v1/ping` returns `{"status":"pong"}`.

Elastic IP note: without one, the instance's public IP changes on stop/start — you'd then update the `EC2_HOST` secret.

14. Quick command reference

Docker

```
docker build -t app:latest .           # build
docker run -d -p 80:8080 --name app app # run
docker ps                             / docker ps -a   # list running / all
docker logs app                       / docker logs -f app
docker exec -it app sh                 # shell inside
docker stop app                       / docker rm app
docker images                         / docker rmi <id>
docker pull <image> / docker push <image>
docker network create net / docker network ls
docker volume ls                      / docker volume inspect <v>
docker image prune -f / docker system prune -f
```

Docker Compose

```
docker compose up -d           docker compose ps
docker compose pull            docker compose logs -f app
docker compose down            docker compose down -v
```

ECR auth

```
aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin <ACCT>.dkr.ecr.ap-south-1.amazonaws.com
aws ecr describe-images --repository-name inventory-management --region ap-south-1
```

EC2 / SSH

```
chmod 400 key.pem
ssh -i key.pem ec2-user@<PUBLIC_IP>
aws sts get-caller-identity --query Account --output text # your AWS account id
```

GitHub Actions (via gh CLI)

```
gh run list --branch feature/dev
gh run watch <run-id> --exit-status
```

15. What's next — Kubernetes (Phase 6)

The same four-service topology gets re-expressed as Kubernetes **pods** on **k3s** (lightweight single-node k8s):

- **Deployment + Service** for the Java app and Redis
- **ConfigMap + Secret** for the config you currently pass as `-e` env vars
- **ingress-nginx** for real nginx pods + an **Ingress** resource
- **imagePullSecret** so k8s can pull from ECR
- Scaling replicas and rolling updates

Everything you learned about images, env-var config, networks (→ Services), and volumes (→ PersistentVolumes) maps directly onto Kubernetes concepts. Compose was the stepping stone; k8s is the destination.